Facultad de Ingeniería

Proyecto - Etapa 2
Universidad Nacional Autónoma de México
Programación Orientada a Objetos
Grupo 7
Fecha de entrega: lunes 26 de octubre de 2026

Simulador de Sistema Hospitalario

Se desea continuar con el desarrollo del simulador de sistema hospitalario, para esta segunda etapa, el sistema deberá evolucionar mediante la aplicación de los conceptos de *encapsulamiento*, *paquetes*, *herencia* y *polimorfismo*, incorporando *clases abstractas* e *interfaces* para representar diferentes tipos de personal médico y pacientes.

El sistema deberá conservar las funcionalidades desarrolladas anteriormente, así como las restricciones establecidas en la primera etapa.

Requisitos del programa

Para esta segunda etapa se consideran los siguientes requisitos:

Encapsulamiento

Todas las clases del sistema deberán aplicar los principios de encapsulamiento, considerando las siguientes restricciones:

- Los atributos de las clases deberán tener visibilidad *private* o *protected* cuando exista una justificación de diseño.
- Implementar métodos de acceso y modificación (*getters* y *setters*) cuando sean necesarios.
- Incorporar mecanismos de validación para impedir que los atributos almacenen valores inválidos.
- Utilizar constructores para inicializar los atributos de los objetos.
- Evitar que las clases externas modifiquen directamente los atributos de los objetos.
- Los estudiantes deberán justificar qué atributos requieren métodos de modificación y cuáles deben permanecer sin posibilidad de modificación después de inicializarse. Por ejemplo, un atributo cédula profesional no necesariamente tendría que contar con un método *setCedula()*.

Paquetes

El programa deberá organizarse mediante paquetes de Java, considerando la responsabilidad de cada conjunto de clases. Se propone el siguiente esquema de referencia:

```
poo/g7/e0/py2/
src/
/hospital/
/sistema/Sistema.java
/personal/
-   Medico.java
-   MedicoGeneral.java
-   MedicoEspecialista.java
-   Enfermero.java
-   EnfermeroEspecialista.java
-   JefeEnfermeria.java
/pacientes/
-   Paciente.java
-   PacienteGeneral.java
-   PacienteEspecialidad.java
/interfaces/
-   AtencionMedica.java (Hacerlo JAR da puntos extra)
test/
doc/
lib/
```

Facultad de Ingeniería

Proyecto - Etapa 2
Universidad Nacional Autónoma de México
Programación Orientada a Objetos
Grupo 7
Fecha de entrega: lunes 26 de octubre de 2026

Se deberá demostrar el uso de *package*, *import* y el uso de *Fully Qualified Names* cuando corresponda. Además, el programa deberá compilarse y ejecutarse respetando la estructura de paquetes.

Herencia

Se deben implementar las siguientes jerarquías de clases:

Paciente(Clase Abstracta/Base)
- PacienteGeneral
- PacienteEspecialidad

Medico(Clase Abstracta/Base)
- MedicoGeneral
- MedicoEspecialista

Enfermero(Clase Abstracta/Base)
- EnfermeroEspecialista
- JefeEnfermeria

AtencionMedica(Interfaz) puede ser un JAR

Las clases derivadas deberán cumplir con lo siguiente:

- Heredar atributos y comportamientos de sus respectivas clases base.
- Incorporar al menos un atributo propio que permita distinguir su especialización.
- Implementar al menos un comportamiento propio.
- Sobrescribir al menos un método heredado cuando su comportamiento sea diferente.
- Usar *super* para invocar constructores o métodos de la clase base.
- Evitar la duplicación innecesaria de atributos y métodos.

Para el caso de los médicos generales y especialistas, por ejemplo, el método *darConsulta()* podrá tener implementaciones diferentes según el tipo de médico.

En caso de los pacientes, *solicitarConsulta()* podrá presentar comportamientos distintos según el tipo de atención requerida.

Polimorfismo

El sistema deberá demostrar el funcionamiento del polimorfismo estático y dinámico.

- Se deberá implementar la sobrescritura de métodos considerando el contexto de clase al que pertenece.
- Se deberán utilizar estructuras de datos que permitan almacenar objetos de diferentes subclases mediante referencias de su clase base o de una interfaz común.
- El método invocado deberá ejecutar el comportamiento al tipo real del objeto.
- Los comportamientos se deberán demostrar en tres jerarquías: pacientes, médicos y enfermeros.

Finalmente, se deberán conservar las restricciones establecidas durante la primera etapa, considerando las siguientes transformaciones:

- Los pacientes generales podrán recibir atención de médicos generales, mientras que los pacientes de especialidad deberán ser asignados a médicos de la especialidad correspondiente.
- Los médicos especialistas solamente podrán atender pacientes de su especialidad, salvo que el diseño contemple explícitamente otra modalidad de atención.
- Los enfermeros especialistas deberán atender pacientes de la especialidad que les corresponda.

- El jefe de enfermería deberá contar con al menos una función adicional de supervisión o coordinación.
- Las estructuras de datos deberán permitir almacenar diferentes tipos de pacientes, médicos y enfermeros sin crear una estructura independiente para cada subclase.
- Los métodos sobrescritos deberán respetar las restricciones generales del sistema hospitalario.
- Se deberán conservar los límites de pacientes y los mecanismos de consulta y tratamiento establecidos en la primera etapa.

Especificaciones de entrega

- El escritor debe elaborar el reporte de los avances de esta etapa del proyecto, explicando las modificaciones realizadas y justificando las decisiones de diseño.
- El editor debe integrar los elementos del reporte, diagramas y evidencias de implementación en la presentación.
- El ponente debe elaborar la presentación para la evaluación presencial, asímismo explicar las jerarquías, interfaces y demostraciones de polimorfismo durante la evaluación presencial.
- El desarrollador debe implementar el código Java, elaborar los algoritmos y pseudocódigos correspondientes a las nuevas funcionalidades y demostrar su ejecución.
- El modelador debe elaborar los diagramas UML actualizados, incluyendo encapsulamiento, herencia, clases abstractas e interfaces.

Se recuerda que también se debe registrar su participación en git, de lo contrario se penaliza en el apartado **Equipo**.

Frequently Asked Questions

- ¿La actividad es en equipo? Si, de acuerdo al rol de cada integrante.
- ¿Puedo usar bibliotecas específicas del lenguaje? No, solo lo que se sugiere en estas especificaciones y lo que se vió en clase durante este parcial.